

Better Lookups for `map` and `unordered_map`

Document #: P3091R2
Date: 2024-05-21 22:11 EDT
Project: Programming Language C++
Audience: LEWGI
Reply-to: Pablo Halpern
<phalpern@halpernwrightsoftware.com>

Contents

- 1 Abstract
- 2 Change Log
- 3 Motivation
- 4 Proposed Feature
 - 4.1 Overview
 - 4.2 Enhancements to `optional::value_or`
 - 4.2.1 Supplying Fewer or More than One Argument
 - 4.2.2 Returning a Type Other Than `mapped_type`
- 5 Before and After Comparisons
- 6 Alternatives Considered
 - 6.1 Names
 - 6.2 Separate `or_construct` function
 - 6.3 Build Get-Value-or-Return-Alternative Functionality Directly into `map`
 - 6.4 Lazy Evaluation of `value_or` Argument
 - 6.5 Free Functions Instead of Members
- 7 Implementation Experience
- 8 Wording
 - 8.1 Feature-Test Macros
 - 8.2 Changes to `optional`
 - 8.3 Changes to `map` and `unordered_map`
- 9 Acknowledgments
- 10 References

§ 1 Abstract

The most convenient way to look up an element of a map or `unordered_map` is to use the index operator, i.e., `theMap[key]`. This operator cannot be used, however, when 1) the container is `const`, 2) the mapped type is not default constructible, 3) the default-constructed value is inappropriate for the context, or 4) the container should not be modified. These limitations often force the user to resort to the `find` member function, which returns an iterator that points to a pair and typically leads to more complex code having at least one `if` statement and/or duplicate lookup operations. Taking inspiration from other languages, especially Python, this paper proposes a `get` member function that returns `optional<T&>` and leverages the observers and monadic operations of `optional` to simplify lookups for map and `unordered_map`. In addition, the `value_or` observer function in `optional` is extended so as to simplify a set of common use-cases for `get`. Note that this proposal is built on `optional<T&>`, which is described in [P2988R4] and is not yet in the C++ Working Draft.

§ 2 Change Log

R2 (2024-05-23 pre-St. Louis mailing)

- LEWG Reached consensus on some aspects of [P2988R4] - specifically that `value_or` should return an rvalue for both `optional<T>` and `optional<T&>`. This change is reflected in the wording for this paper. Rather than a separate `optional<T>::or_construct` member, the functionality of `or_construct` is folded into an enhanced `value_or` for both `optional<T>` and `optional<T&>`.

R1 (post 2024-02-27 LEWGI teleconference)

- As a result of discussions in LEWGI on 27 Feb 2024, the return value of `get` was changed from `mapped_type` to `optional<mapped_type&>`, and the functionality of `get_ref` and `get_as` were delegated to `optional`. Note that `optional<T&>` is not valid in C++23 but is proposed in [P2988R4] for C++26.
- Added a proposed `or_construct` member to `optional`
- Proposed an enhancement of `optional<T&>::value_or` over that in [P2988R4]
- Added feature-test macros

R0 (2024-02-15 pre-Tokyo mailing)

- Initial version

§ 3 Motivation

Just about every modern computer language has, as part of the language or its standard library, one or more associative containers that uniquely map a key to a value, variously called map, hash_map, dictionary, or something similar. In C++, we have `std::map`, `std::unordered_map`, and eventually (per [P0429R9] and [P2767R2]) `std::flat_map`. The easy-to-write and easy-to-read expression for getting a value for an associated key is simply `theMap[key]`; in other words, a mapped value is retrieved (and often set) using the index operator, which returns a reference to the value associated with the key. Unfortunately, the index operator in the C++ associative containers has a number of shortcomings compared to many other languages.

- It works only for non-`const` containers.

- It works only for default-constructible mapped types.
- The default-constructed object (when available) is not always the correct identity for a given use case.
- It modifies the container when the key is absent.

Consider a `std::unordered_map` named `theMap` that maps an integer key to a floating-point value, modeling a sparse array of `double`. If we want to find the largest of the `double` values mapped to the integer keys in the range 1 to 100, we might be tempted to write the following loop:

```
double largest = -std::numeric_limits<double>::infinity();
for (int i = 1; i <= 100; ++i)
    largest = std::max(largest, theMap[i]);
```

If `theMap` is `const`, the loop will not compile. If any of the keys in the range `[1, 100]` are absent from the map, then `theMap[i]` will return a default-constructed `double` having value 0.0, which might or might not be desirable, depending on whether we want to treat missing values as truly having value 0.0 or to ignore them (or, equivalently, to treat them as having value $-\infty$). Finally if, before executing this loop, `theMap` contains only, say, five entries, then at the end of the loop, it will contain at least 100 entries, most of whose values will be zero.

There are alternatives that avoid all these shortcomings but are significantly less elegant and therefore more error prone. For example, the `at` member function looks a lot like `operator[]` and has none of the above shortcomings, but missing keys are handled by throwing exceptions, making this option impractical for situations other than when the key is almost certain to exist. Moreover, a `try-catch` block within a loop is rarely a clean way to structure code:

```
double largest = -std::numeric_limits<double>::infinity();
for (int i = 1; i <= 100; ++i)
{
    try {
        largest = std::max(largest, theMap.at(i));
    } catch (const std::out_of_range&) { }
}
```

The above code would work with a `const` map, would ignore missing elements (rather than treating them as zeros), and would not populate the map with useless entries, but many programmers would argue that the loop is inelegant at best. In most C++ implementations, this code would be extremely inefficient unless key misses are rare.

The other obvious alternative uses the `find` member function:

```
double largest = -std::numeric_limits<double>::infinity();
for (int i = 1; i <= 100; ++i)
{
    auto iter = theMap.find(i);
    if (iter != theMap.end())
        largest = std::max(largest, iter->second);
}
```

This version of the loop is only slightly more verbose than our starting point and avoids all the issues, but using iterators, needing to call *two* member functions (`find` and `end`), and having to extract the second member of the element pair for what should be a *simple* operation increases the cognitive load on both the programmer and the reader. Moreover, a generic use of `find` can yield a subtle bug. Consider the following function template, `f`:

```
template <class Key, class Value>
void f(const Key& k, const std::map<Key, Value>& aMap)
{
    Value obj = some-default-obj-value-expression;
    auto iter = aMap.find(k);
    if (iter != aMap.end())
        obj = iter->second;
    // code that uses `obj` ...
}
```

An instantiation of `f` will not compile unless `Value` is copy assignable. Worse, unless tested with a nonassignable type, the bug could go undetected for a long time. One fix would be initialize `obj` in a single expression:

```
Value obj = aMap.count(k) ? aMap.at(k) : some-default-obj-value-expression;
```

While correct, this solution involves two lookup operations: one for the `count` and one for the `at`. A better fix requires a bit more code:

```
auto iter = aMap.find(k);
Value obj = iter != aMap.end() ? iter->second : some-default-obj-value-expression;
```

Note that the last solution again involves `iterator`, `pair`, and a conditional expression, which is a far cry from the simplicity of `operator[]`.

Let's contrast these less-than-ideal map lookups to dictionary lookups in another language and consider one way to write the largest-value computation in Python:

```
inf = float("inf")
largest = -inf
for i in range(1, 101):
    largest = max(largest, theMap.get(i, -inf))
```

The `get` member of Python's dictionary type looks up the supplied key (first argument). If the key exists in the dictionary, `get` returns the corresponding value; otherwise, `get` returns the specified alternative value (second argument).

In this paper, I propose a `get` member function for `std::map` and `std::unordered_map` similar to the `get` member in Python dictionaries. Because C++ does not have a `None` value like Python's, `get` returns an `optional` instead and delegates the construction of an alternative return value to the `optional` observer members.

§ 4 Proposed Feature

§ 4.1 Overview

What's desired is a simple expression that, given a key, returns the mapped value if the key exists in a specific map and a user-supplied *alternative value* if the key does not exist. The proposed feature is a `get` member function for `map` and `unordered_map` that returns `optional<T>` (or, for a `const` map, `optional<const T>`):

```
template<class Key, class T, class Compare = less<Key>,
        class Allocator = allocator<pair<const Key, T>>>
class map {
    // ...
```

```
optional< mapped_type&> get(const key_type& k);
optional<const mapped_type&> get(const key_type& k) const;
//...
};
```

These functions depend on having an optional template that can be instantiated with reference types, as proposed in [P2988R4].

Using this feature, the earlier example could be rewritten:

```
constexpr double inf = std::numeric_limits<double>::infinity();
double largest = -inf;
for (int i = 1; i <= 100; ++i)
    largest = std::max(largest, theMap.get(i).value_or(-inf));
```

§ 4.2 Enhancements to optional::value_or

To maximize the usefulness and convenience of the proposed get function, I also propose enhancements to optional<T>::value_or, for both reference and non-reference T, as follows (along with const- and reference- qualified overloads and overloads accepting initializer_list).

```
template <class U = remove_cvref_t<T>, class... Args>
    U value_or(Args&&... args);
```

The interface, although significantly extended, is API compatible with the existing standard; i.e., theMap.value_or(x) returns a T object having either the engaged value, if any, or else a T constructed from x. There should be no ABI breakage, as value_or is already a template and taking its address is not supported by the Standard.

The proposed enhancements add two new features to value_or:

- The variadic argument list provides the ability to specify zero or more constructor arguments for the return value, rather than exactly one.
- A return type other than T can be specified explicitly, if desired.

The benefits of these two enhancements are described below.

§ 4.2.1 Supplying Fewer or More than One Argument

Consistent with other emplace-like functions added to the Standard Library since C++11, making value_or variadic eliminates the need to construct an rvalue then move it into place. Instead, the constructor is not invoked unless needed, and the return object is constructed in place. Compare:

Before	After
<pre>// Unconditionally construct argument. // Conditionally move into the result. std::string result = theOpt.value_or(std::string(a, len));</pre>	<pre>// Conditionally construct the // result // from (a, len). std::string result = theOpt.value_or(a, len);</pre>
<pre>// Return the T value at k, or else a // default-constructed T. auto iter = theMap.find(k);</pre>	<pre>// Return the T value at k, or // else a // default-constructed T.</pre>

Before	After
<pre>T result = iter != theMap.end() ? iter->second : T();</pre>	<pre>T result = theMap.get(k).value_or();</pre>

§ 4.2.2 Returning a Type Other Than mapped_type

Although the return type of `value_or` continues to default to `T`, the proposed change allows the user to explicitly specify a different return type. This feature is useful when the desired type is *convertible from* `T` and where a more efficient construction is possible for the alternative value. An emblematic example is `std::string_view`. Constructing a `string_view` directly from a `char` array or directly from an `std::string` is efficient, but converting a `char` array first to a `std::string` and then constructing a `string_view` from the resulting string is inefficient. By specifying `std::string_view` as the return type, `value_or` can be called with `std::string` as `T` and `char[]` as the alternative value, without generating temporary objects:

```
std::optional<std::string> theOpt;
std::string_view sv1 = theOpt.value_or<std::string_view>("empty");

std::map<int, std::string> theMap{ ... };
std::string_view sv2 = theMap.get(key).value_or<std::string_view>("none");
```

The example above creates the resulting `std::string_view` objects either from the `std::string` stored in the optional or from the `const char[]` passed as the alternative value, without creating an intervening `std::string`. Not only would conversion to an intermediate `std::string` be inefficient, such a conversion would create problems with object lifetimes:

```
// Bug (UB): dangling reference constructing `string_view` from temporary `string`
std::string_view sv1 = theOpt.value_or("none");
```

If the specified return type is a reference, then `value_or` can provide (possibly modifiable) access to the engaged value, if any, without making a copy. When returning a reference, the parameter list is restricted to exactly one argument of type convertible to the return type:

```
std::optional<int&> theOpt;
const int zero = 0;

auto v1 = theOpt.value_or(zero);           // OK, makes a copy
auto& v2 = theOpt.value_or<int&>(zero);     // Error: `const` mismatch
auto& v3 = theOpt.value_or<const int&>(zero); // OK
```

Returning an alternative value of modifiable reference type is less common, but useful in certain situations, such as when the modified value is discarded:

```
std::map<std::string, int> theMap{ ... };
// Increment the entries matching `names` but only if they are already in
    `theMap`.
for (const std::string& name : names) {
    int temp = 0;
    ++theMap.get(name).value_or<int&>(temp); // increment through reference
    // Possibly-modified value of `temp` is discarded.
}
```

§ 5 Before and After Comparisons

The following table shows how operations are simplified using the proposed new member functions. In these examples, K, T, and U are object types; m is an object of (possibly `const`) `std::map<K, T>`, `unordered_map<K, T>`, or `flat_map<K, T>`; k is a value compatible with K; v is an lvalue of type (possibly `const`) T; and a1..aN are arguments that can be used to construct an object of type T.

Before	After
<pre>auto iter = m.find(k); T x = iter == m.end() ? T{} : iter->second;</pre>	<pre>T x = m.get(k).value_or();</pre>
<pre>auto iter = m.find(k); T x = iter == m.end() ? T(a1...aN) : iter->second;</pre>	<pre>T x = m.get(k).value_or(a1...aN);</pre>
<pre>auto iter = m.find(k); T& x = iter == m.end() ? v : iter->second;</pre>	<pre>T& x = m.get(k).value_or<T&>(v);</pre>
<pre>map<K, vector<U>> m{ ... }; auto iter = m.find(k); span<U> x = iter == m.end() ? span<U>{} : iter->second;</pre>	<pre>map<K, vector<U>> m{ ... }; span<U> x = m.get(k).value_or<span<U>> ();</pre>
<pre>map<K, vector<U>> m{ ... }; const array<U, N> preset{ ... }; auto iter = m.find(k); span<const U> x = iter == m.end() ? span<const U>(preset) : iter->second;</pre>	<pre>map<K, vector<U>> m{ ... }; const array<U, N> preset{ ... }; span<const U> x = m.get(k).value_or<span<const U>> (preset);</pre>
<pre>unordered_map<K, U*> m{ ... }; if (auto iter = m.find(k); iter != m.end()) { U* p = iter->second; // use p ... }</pre>	<pre>unordered_map<K, U*> m{ ... }; if (U* p = m.get(k).value_or(nullptr); p) { // use p ... }</pre>
<pre>if (auto iter = m.find(k); iter != m.end()) { T& r = iter->second; // use r ... }</pre>	<pre>if (auto q = m.get(k); q) { T& r = *q; // use r ... }</pre>

§ 6 Alternatives Considered

§ 6.1 Names

The name `map::get` is borrowed from the Python dictionary member of the same name. Other names considered were `get_optional`, `lookup`, and `lookup_optional`. The `get` name was selected for brevity and familiarity, but `lookup` is also concise and is a reasonable alternative.

§ 6.2 Separate `or_construct` function

In the R1 draft of this paper, `optional::value_or` was left unchanged and a new function, `or_construct` was proposed. It was pointed out that it is possible to fold `or_construct` into `value_or` without changing any current use of `value_or`, leading to the current design. If the changes to `value_or` are considered problematic in some way, we can revert to having a separate function.

The name `optional<T>::or_construct` was selected for consistency with the monadic operations `and_then` and `or_else`. However, we might consider `value_or_construct` as an intuitive extension of `value_or`.

§ 6.3 Build Get-Value-or-Return-Alternative Functionality Directly into `map`

Version R0 of this paper proposed `get`, `get_ref`, and `get_as` member functions that would look up a key and return the corresponding value (or a reference to the corresponding value) or else construct an alternative from the nonkey arguments without involving `optional<T&>`:

```
template <class... Args>
    mapped_type get(const key_type& key, Args&&... args) const;

// return by reference
template <class Arg>
    common_reference_t<mapped_type&, Arg> get_ref(const key_type& key, Arg&& ref);
template <class Arg>
    common_reference_t<const mapped_type&, Arg> get_ref(const key_type& key, Arg&& ref) const;

template <class U, class... Args>
    U get_as(const key_type& key, Args&&... args);
template <class U, class... Args>
    U get_as(const key_type& key, Args&&... args) const;
```

LEWGI members later pointed out that the `get_as` functionality could be merged into `get` by adding a defaulted `U` parameter to `get`:

```
template <class U = remove_cvref_t<T>, class... Args>
    U get(const key_type& key, Args&&... args);
template <class U = remove_cvref_t<T>, class... Args>
    U get(const key_type& key, Args&&... args) const;
```

The following table shows the usage of the R0 design compared to the currently proposed design.

R0 Design (<code>get</code> Returns <code>T</code>)	Current Design (<code>get</code> Returns <code>optional<T&></code>)
<code>auto tval = theMap.get(k);</code>	<code>auto tval = theMap.get(k).value_or();</code>

R0 Design (get Returns T)	Current Design (get Returns optional<T&>)
<pre>auto tval = theMap.get(k, cT1, cT2);</pre>	<pre>auto tval = theMap.get(k).value_or(cT1, cT2);</pre>
<pre>auto& tref = theMap.get_ref(k, lvalue);</pre>	<pre>auto& tref = theMap.get(k).value_or<T&> (lvalue);</pre>
<pre>auto uval = theMap.get_as<U>(k, cU1, cU2);</pre>	<pre>auto uval = theMap.get(k).value_or<U>(cU1, cU2);</pre>
<pre>if (auto opt = theMap.get_as<std::optional<T&>> (k); opt) { auto& ref = *opt; // ... }</pre>	<pre>if (auto opt = theMap.get(k); opt) { auto& ref = *opt; // ... }</pre>

Advantages of the R0 Design Over the Current Design

- R0 provides more concise and readable calls to `get` and `get_ref` in many cases.
- The desired return-value-if-present-else-return-alternative functionality is all specified in one place in the R0 design, exactly where a beginner would look for it.
- R0 `map` and `unordered_map` specification changes would be independent of `optional<T&>`, thus decoupling this paper from [P2988R4] and its successor [P2988R5].

Advantages of the Current Design Over the R0 Design

- The current `get` provides a direct indication, via a disengaged `optional` return value, that the key was not found.
- By returning `optional`, the current design can leverage all of the functionality of `optional`, including `value_or` and monadic operations. Any future improvements to `optional` could be accessed by users of `map::get` without modifying `map`. In particular, all of the features in [P1255R12] could be applied to the return value of `map::get`, including a (soon to be proposed) `std::reference_or` function.
- The current version has a single observer (`get`) compared to two observers for R0 (`get` and `get_ref`, assuming `get_as` is merged into `get`). Since each observer requires four overloads (`const` and `non-const`, each having a `key_type` or templated `K` parameter), the interface simplification is notable.
- The current specification of `get` is simple to specify and implement, making `get` easy to add to nonstandard map-like containers and new standard containers such as `flat_map`.
- The current interface can be applied not only to associative map-like containers, but to any indexable container, including `vector` and `deque`, should the committee choose to do that.

During the [2024-02-27 LEWGI \(SG18\) telecon](#), unanimous consent was achieved for get returning `optional<T&>` (known as the Alternative Design in [\[P3091R0\]](#)):

POLL: The alternative design with a smaller container API with extensions to `std::optional` should be pursued.

SF	WF	N	WA	SA
6	4	0	0	0

§ 6.4 Lazy Evaluation of `value_or` Argument

The map lookup functionality described in this paper was implemented as free utility functions in [\[Folly\]](#). Specifically, the `folly::map_default(map, key, dflt)` function has similar behavior to the `map.get(key, dflt)` function proposed in R0 of this paper. Folly, however, goes one step further: If `dflt` is a functor taking no arguments and returns a value convertible to `mapped_type`, then the functor is invoked if and only if the key is not found in the map; i.e., the alternative value is computed *lazily* when needed.

If such lazy evaluation is desired within the framework of this proposal, the functionality would be added to `optional<T>::value_or`. Although not directly related to map lookups, I would consider extending this proposal accordingly.

§ 6.5 Free Functions Instead of Members

Providing the functionality described in this paper is possible using namespace-scope functions, without modifying `std::map` and `std::unordered_map`:

```
template <class Map, class K, class... Args>
    typename optional<typename Map::mapped_type&> get(Map& m, const K& k);
template <class Map, class K, class... Args>
    typename optional<const typename Map::mapped_type&> get(const Map& m, const K&
        k);

auto x = get(m, k).value_or(v);
```

One benefit to this approach is that the namespace-scoped `get` template can handle any *map-like* dictionary type (i.e., a type having a `mapped_type` and a `find` method that returns an iterator pointing to a key-value pair). Such an approach, however, has disadvantages.

- A global function is less intuitive because it puts `get` outside of the map interface.
- The short name can cause ADL issues.
- The *map-like* concept is novel and not used elsewhere in the Standard. Whether many classes model this concept and would therefore benefit from the generality is unclear.

§ 7 Implementation Experience

An implementation, with tests and usage examples, can be found at https://github.com/phalpern/WG21-halpern/tree/main/P3091-map_lookup.

Some of the functionality can be found in Meta's [\[Folly\]](#) library.

§ 8 Wording

The changes below clearly note whether they are relative to the 2023-12-18 Working Draft, [N4971], or to the paper proposing optional<T&>, [P2988R4].

§ 8.1 Feature-Test Macros

To the list in 17.3.2 [version.syn]¹, add:

```
#define __cpp_lib_map_get_optional yyyymmL // also in <map>, <multimap>
```

and

```
#define __cpp_lib_optional_variadic_value_or yyyymmL // also in  
<optional>
```

§ 8.2 Changes to optional

These changes to value_or make the facilities in this paper more convenient to use. The mandate preventing accidentally binding a returned reference to a temporary is copied from [P2255R2]. The text description can be replaced by the use of the reference_constructs_from_temporary_v type trait once P2255 is accepted into the working paper.

In 22.5.3.1 [optional.optional.general] in the *Working Draft*, modify the value_or observer:

```
template<class U> constexpr T value_or(U&&) const &;  
template<class U> constexpr T value_or(U&&) &&;  
  
template <class U = remove_cvref_t<T>, class Self, class... Args>  
    U value_or(this Self&& self, Args&&... args);  
  
template <class U = remove_cvref_t<T>, class Self, class X, class...  
Args>  
    U value_or(this Self&& self, initializer_list<X> il, Args&&... args);
```

In the optional<T&> wording in [P2988R4], change value_or in [optional.optional.general]:

```
template<class U> constexpr T value_or(U&&) const;  
  
template <class U = remove_cv_t<T>, class... Args>  
    U value_or(Args&&...) const;  
template <class U = remove_cv_t<T>, class X, class... Args>  
    U constexpr value_or(initializer_list<X>, Args&&...) const;
```

At the end of 22.5.3.6 [optional.observe] in the *Working Draft*, modify the definition of value_or:

```
template<class U> constexpr T value_or(U&&) const &;
```

Description...

```
template<class U> constexpr T value_or(U&&) &&;
```

Description...

```
template <class U = remove_cvref_t<T>, class Self, class... Args>
    U value_or(this Self&& self, Args&&... args);
```

Mandates: `is_constructible_v<U, decltype(*std::forward<Self>(self))>` &&

`is_constructible_v<U, Args...>` is true. If U is a reference type, then `sizeof...(Args)` is 1 and, for v being a value of the single type in Args, the initialization `U u(v);` is well formed and does not bind u to a temporary whose lifetime is extended (6.7.7 [class.temporary]).

Effects: Equivalent to:

```
return self.has_value() ? U(*std::forward<Self>(self)) : U(std::forward<Args>(args)...);
```

```
template <class U = remove_cvref_t<T>, class Self, class X, class...
Args>
    U value_or(this Self&& self, initializer_list<X> il, Args&&... args);
```

Mandates: `! is_reference_v<U>` && `is_constructible_v<U, initializer_list<X>, Args...>` && `is_constructible_v<U, decltype(*std::forward<Self>(self))>` is true.

Effects: Equivalent to:

```
return self.has_value() ? U(*std::forward<Self>(self)) : U(il, std::forward<Args>(args)...);
```

In the optional `<T&&>` wording in [P2988R4], change the definition of `value_or` as follows [optional.observe] :

```
template<class U> constexpr T value_or(U&&) const;
```

Description...

```
template <class U = T, class... Args>
    U value_or(Args&&...args) const;
```

Mandates: `is_constructible_v<U, decltype(**this)>` && `is_constructible_v<U, Args...>`

is true. If U is a reference type, then `sizeof...(Args)` is 1 and, for v being a value of the single type in Args, the initialization `U u(v);` is well formed and does not bind u to a temporary whose lifetime is extended (6.7.7 [class.temporary]).

Effects: Equivalent to:

```
return has_value() ? U(**this) :
    U(std::forward<Args>(args)...);
```

```
template <class U = remove_cv_t<T>, class X, class... Args>
    U constexpr value_or(initializer_list<X> il, Args&&...) const;
```

Mandates: ! is_reference_v<U> && is_constructible_v<U,
initializer_list<X>, Args...> &&
is_constructible_v<U, decltype(**this)> is true.

Effects: Equivalent to:

```
return has_value() ? U(**this) : U(il,  
std::forward<Args>(args)...);
```

§ 8.3 Changes to map and unordered_map

In 24.4.4.1 [map.overview]/2, insert the get element-access members:

```
// 24.4.4.3 [map.access], element access
mapped_type& operator[](const key_type& x);
mapped_type& operator[](key_type&& x);
template<class K> mapped_type& operator[](K&& x);
mapped_type& at(const key_type& x);
const mapped_type& at(const key_type& x) const;
template<class K> mapped_type& at(const K& x);
template<class K> const mapped_type& at(const K& x) const;
optional<mapped_type&> get(const key_type& x) noexcept;
optional<const mapped_type&> get(const key_type& x) const noexcept;
template<class K> optional<mapped_type&> get(const K& x) noexcept;
template<class K> optional<const mapped_type&> get(const K& x) const
noexcept;
```

At the end of 24.4.4.3 [map.access], add these descriptions:

```
optional<mapped_type&> get(const key_type& x) noexcept;
optional<const mapped_type&> get(const key_type& x) const noexcept;
template<class K> optional<mapped_type&> get(const K& x) noexcept;
template<class K> optional<const mapped_type&> get(const K& x) const
noexcept;
```

Constraints: For the third and fourth overloads, the *qualified-id*
Compare::is_transparent is valid and denotes a type.

Preconditions: The expression find(x) is well formed and has well-
defined behavior.

Returns: find(x)→second if find(x) == end() is false, otherwise
nullopt.

Complexity: Logarithmic

In 24.5.4.1 [unord.map.overview]/3, insert the get element-access members:

```
// 24.5.4.3 [unord.map.elem], element access
mapped_type& operator[](const key_type& x);
mapped_type& operator[](key_type&& x);
template<class K> mapped_type& operator[](K&& x);
mapped_type& at(const key_type& x);
const mapped_type& at(const key_type& x) const;
template<class K> mapped_type& at(const K& x);
template<class K> const mapped_type& at(const K& x) const;
optional<mapped_type&> get(const key_type& x) noexcept;
optional<const mapped_type&> get(const key_type& x) const noexcept;
template<class K> optional<mapped_type&> get(const K& x) noexcept;
template<class K> optional<const mapped_type&> get(const K& x) const
noexcept;
```

At the end of 24.5.4.3 [unord.map.elem], add these descriptions:

```
optional<mapped_type&> get(const key_type& x) noexcept;
optional<const mapped_type&> get(const key_type& x) const noexcept;
template<class K> optional<mapped_type&> get(const K& x) noexcept;
template<class K> optional<const mapped_type&> get(const K& x) const
noexcept;
```

Constraints: For the third and fourth overloads, the *qualified-ids* `Hash::is_transparent` and `Pred::is_transparent` are valid and denote types.

Preconditions: The expression `find(x)` is well formed and has well-defined behavior.

Returns: `find(x)→second` if `find(x) == end()` is false, otherwise `nullopt`.

§ 9 Acknowledgments

Thanks to Tomasz Kamiński for pushing me on the `optional<T&>` approach.

Thanks to Steve Downey for working with me to harmonize P2988 with this paper.

Thanks to Lori Hughes for editing support.

§ 10 References

[Folly] Meta. `folly/ folly/MapUtil.h`.

<https://github.com/facebook/folly/blob/323e467e2375e535e10bda62faf2569e8f5c9b19/folly/MapUtil.h#L35-L71>

[N4971] Thomas Köppe. 2023-12-18. Working Draft, Programming Languages — C++.

<https://wg21.link/n4971>

[P0429R9] Zach Laine. 2022-06-17. A Standard `flat_map`.

<https://wg21.link/p0429r9>

- [P1255R12] Steve Downey. 2024-01-16. A view of 0 or 1 elements: `views::maybe`.
<https://wg21.link/p1255r12>
- [P2255R2] Tim Song. 2021-10-14. A type trait to detect reference binding to temporary.
<https://wg21.link/p2255r2>
- [P2767R2] Arthur O'Dwyer. 2023-12-09. `flat_map/flat_set` omnibus.
<https://wg21.link/p2767r2>
- [P2988R4] Steve Downey, Peter Sommerlad. 2024-04-16. `std::optional<T&>`.
<https://wg21.link/p2988r4>
- [P2988R5] Steve Downey & Peter Sommerlad. `std::optional<T&>`.
<http://wg21.link/P2988R5>
- [P3091R0] Pablo Halpern. 2024-02-06. Better lookups for `'map'` and `'unordered_map'`.
<https://wg21.link/p3091r0>
-

1. All citations to the Standard are to working draft N4971 unless otherwise specified. ↩